# Design Report: Distributed Prime Finding Platform

AFS-Like Distributed File System

## 1 Executive Summary

This report presents the design and implementation of a fault-tolerant distributed system for finding prime numbers in large datasets. The system has two components: an AFS-like distributed file system and a distributed prime-finding application.

The file system uses a replicated leader-follower architecture employing the Raft consensus algorithm. The application uses a coordinator-worker model to distribute computation, with workers processing data chunks in parallel.

Fault tolerance is achieved through active-passive replication for the coordinator, Raft consensus for file servers, and consistent global snapshots for crash recovery and task resumption.

**System Scale:** Multiple Workers, Replicated Coordinators, and Replicated File Servers.

**Communication:** gRPC protocol

**Key Algorithms:** Global snapshots, Raft consensus, exponential backoff with failover

## 2 System Model

The system finds unique prime numbers and stores them in a distributed file server. It comprises three process types communicating via gRPC over an asynchronous network with possible crash-fail errors.

**File Servers (Replicated):** A Raft-based replicated cluster for high-availability storage.

**Application Coordinator (Replicated):** A central replicated service managing task assignment and snapshots.

**Application Workers:** A pool of processes performing computation and interacting with File Servers.

## 3 System Assumptions and Limitations

### 3.1 Assumptions

**Network:** Eventual connectivity with transient partitions. Crash-fail model with 30–60 s gRPC timeouts.

**Storage:** Durable POSIX-compliant storage; no silent disk corruption.

**Processes:** Fail-stop failures, independent and uncorrelated.

### 3.2 Limitations

**Byzantine Failures:** Not supported.

**Extreme Delays:** Treated as process failures.

**Concurrent Failures:** Manual intervention required for coordinator or quorum loss.

# 4 Part 1: AFS-Like Distributed File System

The file system provides highly available, read-optimized storage for datasets and results.

## 4.1 Architecture and Core RPCs

The design implements an AFS-like whole-file caching model. Clients fetch the full file on open and perform local reads. Writes are flushed on close.

- `UploadFile(stream FileChunk)`: worker uploads
- `FetchFile(FetchRequest)`: worker downloads
- `RequestVote(VoteRequest)`: Raft leader election
- `Heartbeat(HeartbeatRequest)`: Raft authority maintenance
- `ReplicateFile(stream Req)`: leader data replication to followers

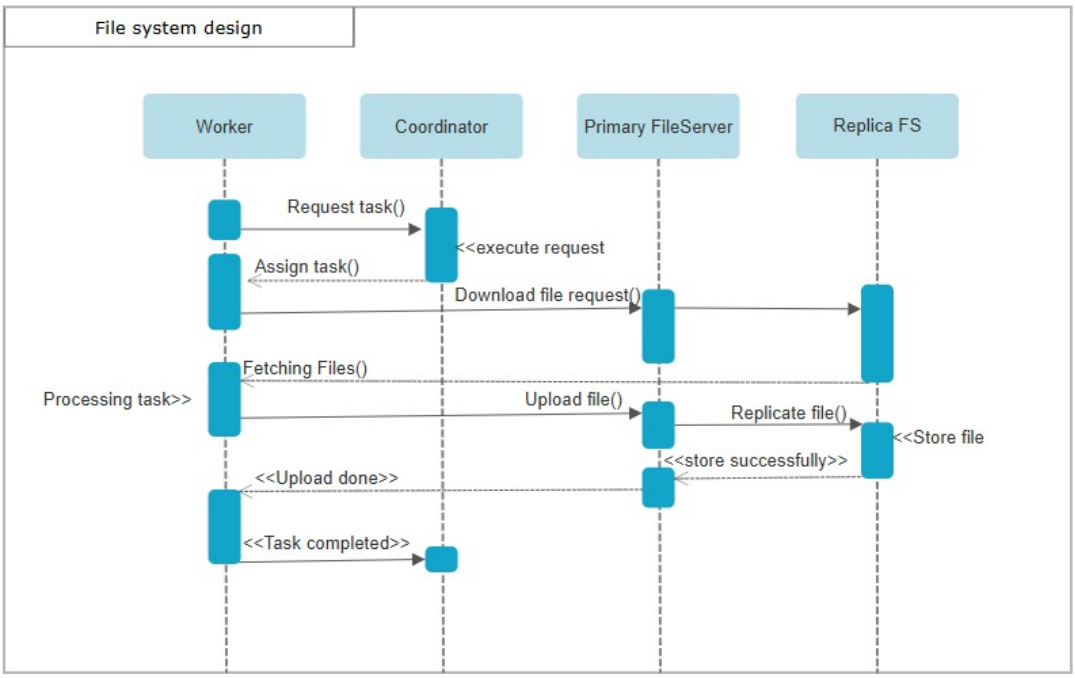



Figure 1: File system design sequence diagram.

## 4.2 File Server Fault Tolerance (Raft)

- **Leader Election:** Servers start as Followers. If no heartbeats are received within a randomized timeout, a server becomes Candidate, requests votes, and if it wins a majority, becomes Leader.
- **Write Path:** Only the Leader handles `UploadFile`, commits data locally, and replicates via `ReplicateFile`.
- **Read Path:** Any server can serve reads, enabling load balancing.

# 5 Part 2: Distributed Prime Finding Application

The application layer manages distributed computation via a Coordinator-Worker model.

## 5.1 Application Fault Tolerance

- **Coordinator Failure:** A backup runs in active-passive mode. On primary failure, workers reconnect to the backup.
- **Worker Failure:** Workers send heartbeats to the Coordinator. Missing heartbeats mark the worker dead and trigger task reassignment.
- **Task Re-assignment:** Failed tasks are reassigned. Workers save progress to the file system as snapshots, enabling resumed computation.

# 6 Failure Handling Summary

Table 1: Failure Types and Recovery Mechanisms

| **Failure Type** | **Detection** | **Recovery** | **Guarantee** |
|---|---|---|---|
| Client/Worker Crash | Alive lock file | Snapshot reload and task resume | Eventually consistent |
| Coordinator Crash | gRPC timeout | Failover to secondary | High availability |
| FileServer Crash | gRPC timeout | Raft leader election | Linearizable consistency |
| Network Partition | Deadline exceeded | Exponential backoff and retry | Eventually consistent |
| Data Corruption | Checksum mismatch | Reject and retry | Strong integrity |
| Task Failure | Heartbeat timeout | Task reassignment | At-least-once semantics |